多智能体驱动的嵌入式代码检测系统
调研及实施方案

项　目　名　称：**嵌入式代码检测系统**

编　制　单　位：**嵌入式代码检测系统项目组**

编　制　依　据：**技术调研（2026 年 9-10 月）**

文　档　版　本：**V1.0**

完　成　日　期：**2026 年 10 月**

摘　要

嵌入式 C 代码检测面临一个双重矛盾：传统静态分析工具在跨函数语义判断上存在盲区且误报率偏高，而具备语义理解能力的大模型又存在幻觉与过度自信问题——已有研究测得大模型生成的 C 代码安全缺陷率可达 49%。本方案围绕这一矛盾展开调研与设计。技术路线部分梳理了大模型参与检测的四条路线（直接检测、检索增强、工具引导的迭代修复、领域微调）及各自代表工作与实验结论；开源生态部分盘点了静态与动态分析工具、数据集资源与大模型加嵌入式的参考项目，并按创建年份与星标数标注影响力；流程设计部分提出置信度触发的自适应检索与工具引导迭代修复相结合的三层检测架构（静态预筛选、检索增强的语义检测、动态验证与反馈），核心设计原则是不把判断权完全交给大模型，用工具反馈约束其输出；最后给出关键挑战（硬件语义建模、高置信度误分类抑制、领域数据集构建）的分析与对策，以及约 8 至 11 周的分阶段实施计划。文中开源项目星标数均为截至 2026 年 10 月的快照数据。

关键词：嵌入式代码检测；大语言模型；静态分析；检索增强生成；多智能体

目　录

摘　要 **2**

第 1 章　背景与问题定义 **3**

1.1 嵌入式 C 代码的特殊性 3

1.2 现有检测手段的覆盖边界与局限 4

1.3 大模型介入的机会窗口与风险 4

1.4 系统定位：在静态与动态之间补缝隙 4

第 2 章　技术路线与开源生态调研 **5**

2.1 四条技术路线 5

2.1.1 路线一：直接检测（Zero-shot / Few-shot） 5

2.1.2 路线二：检索增强（RAG） 5

2.1.3 路线三：工具引导的迭代修复 6

2.1.4 路线四：领域微调（LoRA / QLoRA） 6

2.2 开源静态分析工具 6

2.2.1 CodeChecker 6

2.2.2 Cppcheck 7

2.2.3 Frama-C 7

2.2.4 NaiveSystems Analyze 7

2.2.5 选型补充说明 7

2.3 动态分析工具 8

2.3.1 IAR C-RUN（工业实践案例）.....8
2.3.2 u-INJECTOR（学术前沿）.....8
2.3.3 开源替代路线.....8
2.4 数据集资源.....9
2.4.1 Toyota ITC 基准.....9
2.4.2 EmbedEvalC.....9
2.4.3 Big-Vul.....9
2.4.4 补充：NIST Juliet 测试集.....10
2.5 大模型加嵌入式检测的参考项目.....10
2.5.1 LCMHalMCP.....10
2.5.2 embedded-ai-workspace.....10
2.5.3 中科院软件所嵌入式漏洞挖掘技术.....11
2.5.4 补充：通用对照基线.....11
第 3 章 检测流程设计.....11
3.1 传统检测工作流.....12
3.2 两种大模型增强模式.....12
3.2.1 模式 A：置信度触发的自适应检索（A-RAG）.....12
3.2.2 模式 B：工具引导的迭代修复.....12
3.3 本系统三层架构.....13
3.3.1 第一层：静态预筛选.....13
3.3.2 第二层：RAG 增强的语义检测.....13

3.3.3 第三层：动态验证与反馈.....14

3.3.4 工程要点.....14

第 4 章　关键挑战与实施计划.....14

4.1 关键挑战与对策.....14

4.1.1 嵌入式硬件语义的建模.....15

4.1.2 高置信度误分类的抑制.....15

4.1.3 领域数据集的构建.....15

4.1.4 其他工程风险.....15

4.2 分阶段实施计划.....16

4.3 数据时效性与核实约定.....16

注：本目录由域代码生成。编辑文档后，请在目录上单击右键选择“更新域”刷新页码。

第 1 章　背景与问题定义

本章回答为什么要做这个系统。落脚点在两处：嵌入式代码检测的特殊性，以及大模型介入的动机与边界。

1.1 嵌入式 C 代码的特殊性

嵌入式 C 代码运行在资源受限硬件上，直接操作寄存器和内存，多数目标平台没有操作系统的保护机制。这一根本约束派生出四个具体特点。

第一，内存错误静默化。没有内存管理单元（MMU）的裸机平台上，越界写不触发段错误，而是静默破坏相邻任务的数据与栈，缺陷往往在远端现场才暴露，复现与定位成本极高。

第二，并发与中断上下文交织。中断服务程序（ISR）与任务共享变量、RTOS 多任务竞争、优先级反转等问题高发，且 volatile 不提供原子性保证，多字节共享数据可能读到一半被改写。ISO 26262:2018 已在编码指南中新增并发主题，MISRA C:2023 修订亦补充多线程规则，说明该维度已成规范共识。

第三，硬件强耦合。代码行为依赖具体芯片的外设寄存器、时序与中断向量，每款芯片对应一份上千页的参考手册，主机环境难以复现真实外设行为，导致动态检测输入激励构造困难。

第四，安全标准强约束。汽车（ISO 26262）、航空（DO-178C）、工业（IEC 61508）、军品（GJB 8114/5369）等标准对代码级验证活动有明确且随安全等级递增的强制要求：静态分析在高等级均为高度推荐，结构覆盖率最高要求到 MC/DC（修正条件判定覆盖）。检测不仅是质量活动，更是合规证据链。

1.2 现有检测手段的覆盖边界与局限

静态检测的盲区。以规则匹配与数据流分析为核心的静态工具（Cppcheck、clang-tidy 等）对单函数内的规则违例覆盖良好，但在三处存在系统性盲区：跨函数与跨文件的语义

传播路径；中断上下文与任务上下文之间的数据流；函数指针回调链路。同时误报率偏高，工业实践中告警分诊消耗大量专家时间，成为检测产能的主要瓶颈。

动态检测的限制。AddressSanitizer、ThreadSanitizer、Valgrind 等运行时检测工具依赖操作系统环境，在裸机平台不可用（仅 UBSan 提供可裁剪的 standalone 运行时）；测试激励难以构造、时序不可复现，使动态检测在嵌入式场景的覆盖率与可重复性都受限。

1.3 大模型介入的机会窗口与风险

机会。大语言模型具备跨函数语义理解能力，恰好覆盖静态工具的盲区：其一，跨函数与跨文件的缺陷传播推断；其二，告警的解释与分级（把工具的规则编号翻译成开发者可理解的因果链）；其三，MISRA 中意图类条款的判断（部分规则需理解设计意图而非单纯语法）；其四，单元测试用例生成。

风险。大模型存在幻觉与过度自信问题，不能作为可信的最终裁判。**已有研究测得大模型生成的 C 代码安全缺陷率可达 49%**；2025 年针对 MISRA 合规的实测表明，没有一个主流模型能直接生成完全合规的代码；在无源码的二进制（反编译）场景，模型表现显著弱于源码场景。换言之，大模型的语义理解是真实的能力，但其输出必须经过外部验证才能采信。

1.4 系统定位：在静态与动态之间补缝隙

传统工具不够用、大模型不可全信，这一张力即本系统设计的出发点。系统定位为在静态与动态检测之间补缝隙，而非替代任何一方：静态工具继续负责规则明确、可精确判定的部分（查得全、成本低）；大模型负责静态工具的盲区（跨函数语义、意图判断）并承担告警解释与分级；动态验证（编译、符号执行、仿真）负责给大模型的输出兜底。三者的分工边界在第三章流程设计中具体展开。

第 2 章　技术路线与开源生态调研

本章为全文篇幅最大的一章，分五部分：大模型参与检测的四条技术路线、开源静态分析工具、动态分析工具、数据集资源、大模型加嵌入式的参考项目。每个开源项目均标注创建年份与星标数作为影响力参考。

关于数据时效性：文中星标数为截至 2026 年 10 月的快照，以 GitHub 仓库页面为最终依据，提交正式报告前应重新核实。学术工具（如 Frama-C、u-INJECTOR）与数据集（如 Toyota ITC）星标数不高，是因为其用户群为学术研究者而非大众开发者，其影响力主要体现在学术引用与工业采用上，不能直接以星标数判断技术价值。

2.1 四条技术路线

2.1.1 路线一：直接检测（Zero-shot / Few-shot）

不引入外部组件，直接以提示词驱动大模型判断代码是否存在缺陷。该路线接入成本最低，适合作为基线。但其能力上限受制于两点：上下文窗口内的信息不足以覆盖跨文件工程；无外部反馈约束，幻觉不可检测。ICSE 2025 的系统性实证研究（Vulnerability Detection with Code LMs: How Far Are We?）结论是：代码大模型在真实漏洞检测上远未达到宣传水平，直接检测不宜作为主力路线。

2.1.2 路线二：检索增强（RAG）

检索外部知识（规范条款解释、芯片手册、历史缺陷案例）辅助大模型判断。代表工作是置信度触发的自适应检索 A-RAG：模型先做初步判断，高置信度直接输出，低置信度触发知识库检索后重新判断，在 Toyota ITC 与 Big-Vul 统一数据集上取得 6.7% 至 18% 的 F1 分数提升，代码与评估套件已开源（仓库 Ayanami0730/arag）。该路线的优势是可将嵌入式领域知识（MISRA 条款、寄存器语义）以知识库形式注入而无需重新训练模型；局限是检索质量依赖知识库构建与分块策略，且置信度校准本身是难点（见第 4 章）。

2.1.3 路线三：工具引导的迭代修复

以外部工具的反馈信号驱动大模型迭代修正：编译诊断、静态分析（CodeQL）、符号执行（KLEE）的结果回传给模型，模型据此修改代码或修正判断，再进入下一轮验证。代表工作在 5000 个 C 编程任务上，将 CodeLlama 7B 生成代码的安全缺陷率从 49% 降至 19%。该路线的本质是以可验证的外部信号约束模型输出，可信度最高；代价是链路长单任务多次调用的推理成本高，适合修复与验证环节而非全量扫描。

2.1.4 路线四：领域微调（LoRA / QLoRA）

以参数高效微调使开源模型适配嵌入式场景。代表工作：LLMxCPG（USENIX Security 2025）先以 Joern 代码属性图抽取最小相关切片，再对 Qwen2.5-Coder-32B 做 LoRA 微调，在 C/C++ 漏洞检测上取得优于通用模型的效果；另一项实证显示，以静态分析器反馈闭环引导生成，可使大模型生成代码的 MISRA 违规减少约 83%。该路线增益稳定，但需要训练资源、高质量标注数据，且模型更新滞后于基座迭代，适合作为二期演进选项。

表 2-1 四条技术路线对比

路线	核心方法	代表工作与结论	适用环节
直接检测	提示词直判，无外部组件	ICSE 2025 实证：远未达宣传水平	基线对照
检索增强（RAG）	低置信触发知识检索后重判	A-RAG：F1 提升 6.7%-18%（已开源）	语义研判（本系统主力）
工具引导迭代修复	编译/静态/符号执行反馈驱动修正	5000 任务将缺陷率 49% 降至 19%	修复与验证
领域微调	LoRA/QLoRA 适配嵌入式语料	LLMxCPG（USENIX Sec 2025）；MISRA 违规减 83%	二期演进

路线选择判断：本系统采用路线二加路线三的组合——研判环节用 RAG 增强（覆盖面与成本的平衡点），修复与验证环节用工具反馈（可信度保障）；路线一作为评测基线；路线四列为二期可选。

2.2 开源静态分析工具

2.2.1 CodeChecker

Ericsson 开源，2015 年 6 月创建，目前约 2,250 至 2,600 星。作为 Clang Static Analyzer 与 Clang Tidy 的封装层，提供缺陷数据库与 Web 查看器，适合作为工具链编排层。其对本项目的主要价值不在检测能力本身，而在于将多种分析工具的输出统一管理（存储、去重、标注、报告），这与后续第三章的多层过滤架构中告警池的设计直接对应，可作为参考实现。

2.2.2 Cppcheck

2009 年 3 月创建，目前约 6,700 至 6,800 星，经过近二十年发展，有 300 余名贡献者与 27,000 余次提交。嵌入式场景中最实用的免费开源选项：支持 --platform=arm32-unknown- 等嵌入式平台参数（影响整型宽度等平台相关判断的准确性），内置 MISRA 检查插件（开源版覆盖 MISRA C:2012，规则文本需自备），许可证 GPLv3。其成熟度、社区规模与嵌入式平台感知能力，是本项目静态预筛选层的主力工具。

2.2.3 Frama-C

CEA List 与 INRIA 联合开发，2008 年首次公开发布，GitHub 主仓库约 152 星。采用插件架构，集成抽象解释（Eva 插件，对变量取值范围与运行时错误给出保守但无误报界的分析）与演绎验证（WP 插件，基于 ACSL 契约的数学证明），已被用于航空嵌入式 C 代码的形式化验证。需要特别说明：其星标数远低于工程类工具，是因为学术工具的用户群与使用方式不同，不能以星标数判断其技术价值——在需要数学级保证的高危模块验证场景，它是开源领域几乎唯一的选择。

2.2.4 NaiveSystems Analyze

2023 年 11 月首次开源发布，目前约 177 星，采用 GNU GPL v3 协议。支持 MISRA C:2012 第三版检查，定位是 Polyspace、Helix QAC 等商业工具的 vendor-neutral 替代品。项目较新、星标不高，但填补了开源 MISRA 检查工具的空白，其规则实现可作为自建规则库的参照。

2.2.5 选型补充说明

除上述四者外，clang-tidy 与 GCC -fanalyzer 随编译工具链自带、无部署成本，应纳入预筛选层。工程实现上的关键建议：以 SARIF（OASIS 静态分析结果交换标准）作为统一告警格式——Cppcheck 2.15 起、GCC 13 起原生支持，CodeChecker 可将 Clang 系输出统一管理，多工具告警归一后，后续研判与报告层只需实现一种数据模型。

表 2-2 开源静态分析工具汇总（星标数截至 2026 年 10 月）

工具	创建时间	星标数	许可证	在本系统中的角色
Cppcheck	2009 年 3 月	约 6,700-6,800	GPLv3	静态预筛选主力（平台感知 +MISRA 插件）
CodeChecker	2015 年 6 月	约 2,250-2,600	Apache-2.0	多工具输出统一管理与告警池参考
Frama-C	2008 年	主仓库约 152	LGPL-2.1	高危模块形式化验证（后备）
NaiveSystems Analyze	2023 年 11 月开源	约 177	GPLv3	开源 MISRA 检查空白填补，规则参照
clang-tidy / GCC -fanalyzer	随 LLVM / GCC	—	Apache-2.0 / GPL-3.0	预筛选层零成本补充

2.3 动态分析工具

2.3.1 IAR C-RUN（工业实践案例）

商业工具，不纳入选型，但其产品形态值得分析：以 IDE 配置、重新构建、调试运行三步流程将运行时检测（越界、溢出、栈检查）融入日常开发，几乎没有使用门槛。这提示本系统的动态验证层应同样以低门槛方式嵌入既有构建流程，而非要求开发者切换到独立环境。

2.3.2 u-INJECTOR（学术前沿）

2025 年发表的研究成果，基于 Unicorn CPU 虚拟化实现固件动态分析：自动分析可执行文件的符号并建立虚拟化环境，相比 QEMU 显著降低了环境构建成本。作为学术工具，星标数不是衡量指标，但其思路——以 CPU 级仿真替代全系统仿真、自动化构建执行环境——对本系统第三层动态验证的裸机目标覆盖有直接参考价值。

2.3.3 开源替代路线

开源侧的务实组合：宿主机上以 CMock 对硬件抽象层打桩、运行 Unity 单元测试（Ceedling 串联构建与覆盖率统计）；需要外设行为时使用 Renode 做外设级仿真（MIT 许可）；裸机上唯一可用的运行时检测是 UBSan standalone。该组合与 u-INJECTOR 思路互补，共同构成第三层的技术储备。

表 2-3 动态分析工具与实践

工具/实践	类型	关键点	对本系统的价值
IAR C-RUN	商业	三步流程融入日常开发	低门槛嵌入的产品形态参照
u-INJECTOR（2025）	学术	Unicorn 虚拟化，自动建环境，成本低于 QEMU	第三层裸机验证的技术储备
CMock + Unity + Ceedling	开源（MIT）	主机侧打桩运行单元测试	第三层首选实现路径
Renode	开源（MIT）	外设级多节点仿真	外设行为模拟
UBSan standalone	开源	裸机唯一可用的运行时检测	裸机动态检测兜底

2.4 数据集资源

2.4.1 Toyota ITC 基准

2015 年 2 月创建，GitHub 仓库约 113 星。包含真实嵌入式 C 程序与精心设计的缺陷覆盖 51 种缺陷子类型，归为 9 个高层类别。这是嵌入式代码缺陷检测研究中引用最广的基准数据集。星标数不高是因为它作为静态分析评测数据，用户群集中在学术研究者而非大众开发者——其影响力体现在被大量论文作为标准评测集引用。

2.4.2 EmbedEvalC

滑铁卢大学 2025 至 2026 年提出，专门用于评估大模型生成的嵌入式 C 代码质量。其研究结论对本项目有直接引用价值：大模型生成的代码在需要复杂内存交互的任务上表现较差——这从数据集侧再次印证了第 1 章的判断（大模型不可全信，内存相关语义是其弱项，恰是嵌入式的高危区）。

2.4.3 Big-Vul

从真实开源项目提取的漏洞与修复函数对数据集，覆盖多种 CWE 类别，规模大、来源真实。但其在嵌入式场景的针对性不如 Toyota ITC（来源项目以通用软件为主，不含硬件约束上下文），适合作为扩充语料与泛化评测，而非主基准。

2.4.4 补充：NIST Juliet 测试集

美国国家标准与技术研究院维护的 C/C++ 测试集，包含数万个按 CWE 注入缺陷的现成用例，覆盖面广但语义深度浅（多为人工构造的短函数）。建议与 Toyota ITC 互补使用 Juliet 用于评测工具与模型的覆盖率（广度），Toyota ITC 用于评测真实程序语义下的检出与分诊能力（深度）。

表 2-4 数据集资源汇总

数据集	时间	内容与规模	适用场景
Toyota ITC	2015 年 2 月	真实嵌入式 C 程序，51 缺陷子类 / 9 大类	主基准（真实语义，深度评测）
EmbedEvalC	2025-2026	评估 LLM 生成嵌入式 C 代码质量	生成代码质量评测与结论引用
Big-Vul	—	真实项目漏洞/修复函数对，多 CWE	扩充语料与泛化评测
NIST Juliet	持续维护	数万个 CWE 注入缺陷用例	覆盖广度评测（与 ITC 互补）

2.5 大模型加嵌入式检测的参考项目

2.5.1 LCMHalMCP

作者 fireknight-hJ，GitHub 开源。将 CodeQL 静态分析、MCP 工具接口、LangGraph 多智能体编排、源码替换、交叉编译与 fuzzemu/Unicorn 仿真串成闭环，支持 STM32、NXP i.MX RT、Zephyr 等平台，以及裸机、FreeRTOS、RT-Thread 等运行时。这是理解工具链如何与大模型协同的最佳开源参考：其架构把检测（CodeQL）、决策（多智能体）、验证（仿真）分层组织，与本项目三层架构的思路同构，其架构图值得在技术选型阶段逐层对照分析。

2.5.2 embedded-ai-workspace

作者 atillab1，2026 年 5 月创建，在 STM32F4-DISCOVERY 开发板上完成了端到端验证。展示了三阶段 Claude 工具链设计：MISRA Skill（纯推理层，不运行代码）、Datasheet MCP（对 1700 页 RM0090 参考手册做向量检索）、Serial MCP（USB UART 硬件交互）。该项目最关键的设计洞察是 Skill 与 MCP 的分工——Skill 改变模型的行为方式，MCP 扩展模型的能力边界。这一区分对本系统设计大模型的参与形式有直接启发：MISRA 条款判断这类稳定的行为模式适合沉淀为 Skill（提示词层），芯片手册检索与硬件交互这类能力扩展适合实现为 MCP 工具层。

2.5.3 中科院软件所嵌入式漏洞挖掘技术

目前处于中试阶段。采用多层次上下文提示结合 AST 抽取语义信息，实现片段、模块、项目三级缺陷检查；修复阶段采用分析、生成、验证、优化四步流程。其三级检查的粒度划分（片段到项目）与本系统静态预筛选到 RAG 研判的分层思路可互相印证。

2.5.4 补充：通用对照基线

alibaba/open-code-review（2026 年 9 月实测约 42,795 星，Apache-2.0）是通用 AI 代码审查的代表，虽非嵌入式特化，但可作为效果对照基线：本系统的评测应证明其在嵌入式场景上优于通用方案，差异化才有说服力。

表 2-5 大模型加嵌入式检测参考项目汇总

项目	定位	关键设计	对本系统的启发
LCMHalMCP	检测-决策-验证闭环	CodeQL+MCP+LangGraph+交叉编译+Unicorn 仿真；多平台多 RTOS	三层架构的同构开源参考
embedded-ai-workspace	硬件端到端验证	MISRA Skill + Datasheet MCP + Serial MCP	Skill 与 MCP 的分工原则
中科院软件所（中试）	三级缺陷检查	多层次上下文+AST；分析-生成-验证-优化四步修复	检查粒度划分与修复流程
open-code-review（对照）	通用 AI 审查	确定性流水线+LLM 智能体混合	评测对照基线

第 3 章　检测流程设计

本章回答大模型怎么嵌入。先给出传统检测的标准工作流，再对比两种经过实证的大模型增强模式，最后给出本系统的三层架构设计。

3.1 传统检测工作流

静态检测标准流程：预处理与编译警告、规则与数据流检查、告警产生、人工分诊与修复，各环节以报告形式交接；动态检测标准流程：构建目标程序、部署到目标环境（真机或仿真）、施加测试激励、观测运行时行为、统计覆盖率。两条流水线相互独立，交接点依赖人工，主要损耗在告警分诊（静态侧）与激励构造（动态侧）。大模型的嵌入点即这两个损耗处。

3.2 两种大模型增强模式

3.2.1 模式 A：置信度触发的自适应检索（A-RAG）

核心机制：大模型对告警或代码先做初步判断并给出置信度；高置信度直接输出结论，低置信度触发知识库检索（规范条款、手册、相似案例），携带检索结果重新判断。该模式以最小代价注入领域知识，在 Toyota ITC 与 Big-Vul 统一数据集上实现 6.7% 至 18% 的 F1 分数提升，代码与评估套件已在 GitHub 开源（仓库 Ayanami0730/arag），可直接复用其实现与评估方法。其命门在置信度校准：若模型对错误判断给出高置信度，检索环节将被跳过，错误直达输出（该风险在第 4 章展开）。

3.2.2 模式 B：工具引导的迭代修复

核心机制：编译诊断、CodeQL 静态分析、KLEE 符号执行的反馈依次回传，大模型整合反馈后生成修复，再进入下一轮验证，直到通过或达到轮次上限。代表性实验在 5000 个 C 编程任务上，将 CodeLlama 7B 生成代码的安全缺陷率从 49% 降至 19%。该模式把判断标准交给可执行的工具信号，可信度最高，适合修复建议的生成与确认环节；但链路长、单任务多次调用，不适合全量扫描。

表 3-1 两种增强模式对比

维度	模式 A（A-RAG 自适应检索）	模式 B（工具引导迭代修复）
核心机制	低置信触发检索，携带知识重判	编译/静态/符号执行反馈驱动迭代
实证效果	F1 提升 6.7%-18%（Toyota ITC+Big-Vul）	缺陷率 49% 降至 19%（5000 任务）
单任务成本	低至中（部分任务免检索）	高（多轮工具调用）
可信度保障	依赖置信度校准	由可执行工具信号保证
适用环节	告警研判、语义检测	修复生成、结论验证

结论：研判用 A、修复与验证用 B。两种模式分别匹配检测流水线中两个损耗点的性质：告警分诊量大、单条浅，适合低成本的模式 A；修复验证量小、要求可信，适合高成本的模式 B。

3.3 本系统三层架构

综合第 2 章工具调研与 3.2 节模式对比，系统采用三层设计，数据流见图 3-1。

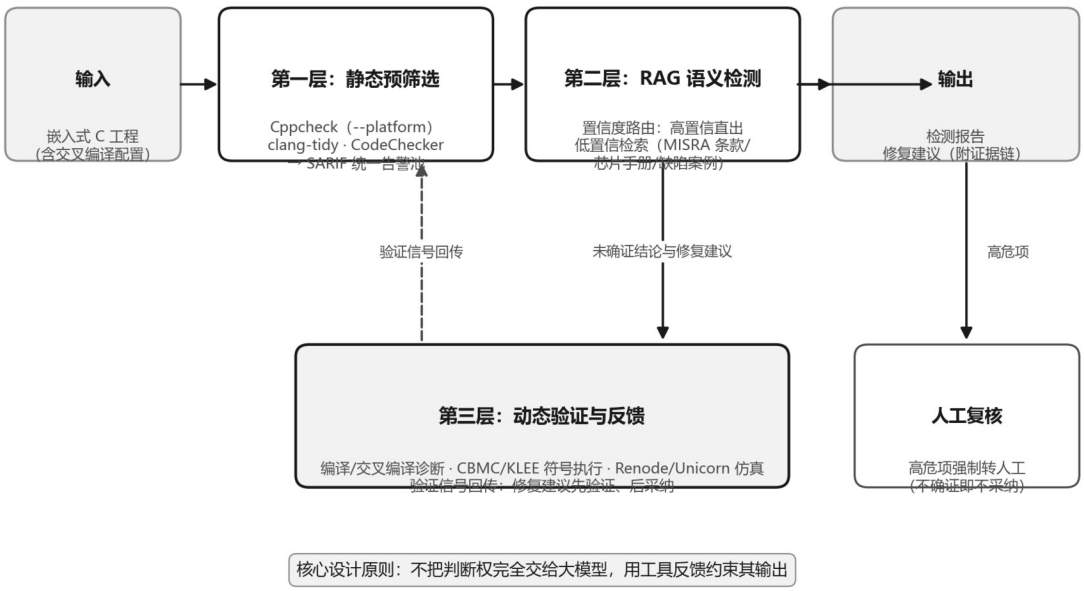


图 3-1 三层检测架构与反馈回路

3.3.1 第一层：静态预筛选

Cppcheck（启用嵌入式平台参数）与 clang-tidy/GCC -fanalyzer 快速过滤规则明确、可精确判定的违例，产出统一为 SARIF 格式的告警池；CodeChecker 的存储与去重机制作为告警池设计参考。该层追求高吞吐、低成本，拦截明确问题，将需要语义判断的疑难告警与跨函数疑点送入第二层。

3.3.2 第二层：RAG 增强的语义检测

对第一层送入的告警与疑点执行模式 A 流程：置信度路由——高置信度直接输出判定低置信度触发检索，知识库包括 MISRA/GJB 条款解释、芯片手册（参照 embedded-ai-workspace 的 Datasheet MCP 实践，对参考手册做向量检索）与历史缺陷案例。该层承担静态工具的盲区（跨函数上下文、中断数据流）与意图类条款判断，每条结论强制附代码证据链（文件、行号、相关调用路径），高危项转人工复核。

3.3.3 第三层：动态验证与反馈

对第二层产出且未确证的结论与修复建议执行模式 B 流程：编译与交叉编译诊断、CBMC/KLEE 符号执行、Renode 或 Unicorn 仿真（参考 u-INJECTOR 思路）依次回传验证信号；验证通过的建议才允许采纳，验证信号同时回传第二层用于修正判定。

核心设计原则：不把判断权完全交给大模型，用工具反馈约束其输出。三层中每一层都有独立于大模型的判据：第一层是规则引擎，第三层是可执行信号，大模型只在第二层发挥语义优势，且其输出受第一层输入约束、第三层验证兜底。该原则直接回应第 1 章的 49% 缺陷率风险。

3.3.4 工程要点

其一，统一数据模型：全链路以 SARIF 为告警与结论的统一格式，证据链作为扩展字段；其二，成本分层：低危告警仅走第一层规则过滤，不过大模型，并设单项目成本上限；其三，可追溯：每次研判记录模型版本、提示词版本与检索命中，保证结论可复现、可回归。

第 4 章　关键挑战与实施计划

4.1 关键挑战与对策

4.1.1 嵌入式硬件语义的建模

难点：寄存器地址、中断向量等硬件语义缺乏机器可读表示，大模型难以正确判断涉及外设交互的代码。对策：其一，向量检索路线——参照 embedded-ai-workspace 对 RM0090 手册（1700 页）的检索实践，已验证可行；其二，结构化注入——CMSIS-SVD 是芯片厂商广泛提供的寄存器描述标准格式（XML），可直接解析为结构化上下文，比纯文本检索更精确，是本系统的扩展方向；其三，LCMHalMCP 的多平台实践（STM32、i.MX RT、Zephyr）证明硬件语义注入在多平台上可复用。

4.1.2 高置信度误分类的抑制

难点：大模型对错误判断常给出高置信度，导致模式 A 的检索分支被跳过、错误直达输出——这是过度自信问题的工程化表现。对策四条：证据链强制（无证据不输出）；置信度校准（以标注集校准阈值，或以多次采样的一致性替代自报置信度）；拒答机制（不确定即转人工，宁可漏报给人工也不给出错误结论）；分层复核（高危项无条件人工确认）。

4.1.3 领域数据集的构建

难点：现有数据集对嵌入式硬件约束的覆盖不足（Big-Vul 来源于通用软件；Juliet 语义深度浅）。对策：以 Toyota ITC 为主基准（真实嵌入式程序、51 缺陷子类），Juliet 补覆盖广度；在其上自建注入缺陷集——在真实开源工程（如 FreeRTOS 示例）中按 Toyota ITC 缺陷分类注入变体，形成带标注的回归集；EmbedEvalC 的结论（模型在复杂内存交互任务上弱）应转化为评测集中内存交互类用例的加权。

4.1.4 其他工程风险

成本：多智能体约 15 倍 token 成本乘数，以分层路由（低危不过模型）与轻量模型分诊对冲；许可证：AGPL/GPL 项目（如 VulnHuntr、EMBA）只借鉴架构、不引入代码，

避免开源传染；评测回归：每次提示词或检索策略改动必须跑全量回归，防止局部改善全局劣化。

表 4-1 挑战与对策汇总

挑战	风险表现	对策
硬件语义建模	外设交互代码误判	手册向量检索（已验证）+ CMSIS-SVD 结构化注入
高置信度误分类	错误结论跳过检索直达输出	证据链强制；置信度校准；拒答转人工；高危人工复核
领域数据集缺失	评测不反映嵌入式真实能力	Toyota ITC 主基准 + Juliet 广度 + 自建注入缺陷集
成本与许可证	规模化不可负担；传染风险	分层路由与成本上限；AGPL/GPL 只学不引

4.2 分阶段实施计划

总周期约 8 至 11 周，分四阶段推进，每阶段以可验收的文档或系统产出收口。

表 4-2 分阶段实施计划

阶段	周期	主要内容	产出与验收标准
一、文献精读与缺陷分类建模	1-2 周	精读四条路线代表工作（A-RAG、工具引导修复、LLMxCPG 等）；建立三层缺陷分类体系（通用 C 缺陷 / 嵌入式特有缺陷 / MISRA 合规性）及其与工具检查项的映射	缺陷分类文档与检测项映射表（评审通过）
二、技术选型与数据准备	2-3 周	跑通 Cppcheck 平台参数与 CodeChecker 告警管理；下载并统一 Toyota ITC、Juliet、Big-Vul 格式；部署 A-RAG 开源实现并完成小规模试跑	工具链验证记录；就绪的评测数据集；A-RAG 试跑报告
三、原型系统搭建	3-4 周	三层架构最小闭环：静态预筛选 + RAG 语义检测 + 编译反馈；先在 Toyota ITC 子集调通，再迁移真实开源工程	可运行原型；子集上端到端演示
四、评估与迭代	2 周	四项指标评测（F1、去误报率、漏判率、单告警成本）；与直接检测基线及通用工具对照；回归脚本化	评估报告与下一迭代计划

计划的弹性说明：阶段一与阶段二可部分并行（文献精读与工具试跑互不阻塞）；阶段三的反馈循环先以编译诊断为最小集（成本最低的验证信号），符号执行与仿真按需引

入；若阶段四评估显示 RAG 增益不显著，回退方案是强化第一层规则库并扩大人工复核范围，系统骨架不受影响。

4.3 数据时效性与核实约定

文中全部星标数为截至 2026 年 10 月的快照数据，仅作影响力参考；开源项目星标数随时间动态变化，提交正式报告或论文前应以 GitHub 仓库页面为最终依据重新核实。对学术工具（Frama-C、u-INJECTOR）与数据集（Toyota ITC），报告中已按其学术引用与工业采用说明实际影响力，避免以星标数误判技术价值。